# LAPORAN PRAKTIKUM PEMROGRAMAN BERBASIS OBJEK

**Minggu Ke-6**

**disusun Oleh:**

**Az Zahrand Solichul Tajussalathin**

**NIM 2511532001**

**Dosen Pengampu: Nurfiah S.ST, M.Kom**

**Asisten Praktikum: Siti Fadhilah Rahmi**




**FAKULTAS TEKNOLOGI INFORMASI**

**DEPARTEMEN INFORMATIKA**

**UNIVERSITAS ANDALAS**

**2026**

# DAFTAR ISI

**DAFTAR ISI** ..... i

**KATA PENGANTAR** ..... ii

**BAB I PENDAHULUAN** ..... 1

**1.1 Latar Belakang** ..... 1

**1.2 Tujuan Praktikum** ..... 1

**1.3 Manfaat Praktikum** ..... 1

**BAB II PEMBAHASAN** ..... 1

**2.1 Implementasi Latihan Abstraction** ..... 1

**2.2 Pemeriksaan Latihan dan Error Compiler** ..... 1

**BAB III TUGAS** ..... 1

**3.1 Evolusi API dan Konflik Diamond** ..... 1

**3.2 Sistem Notifikasi Pesanan** ..... 1

**3.3 Code Review dan Refactor BadShop** ..... 1

**3.4 Refleksi Pilihan Desain** ..... 1

**BAB IV KESIMPULAN** ..... 1

**4.1 Ringkasan Hasil Praktikum** ..... 1

**4.2 Link Github** ..... 1

**DAFTAR PUSTAKA** ..... iii

# KATA PENGANTAR

Puji syukur kehadirat Allah SWT atas limpahan rahmat dan karunia-Nya sehingga laporan praktikum ini dapat disusun dan diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pelaksanaan kegiatan praktikum pada mata kuliah Pemrograman Berbasis Objek.

Ucapan terima kasih disampaikan kepada Bapak Nurfiah S.ST, M.Kom. selaku dosen pengampu, serta kepada asisten laboratorium yang telah memberikan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penulisan maupun isi. Oleh karena itu, saran dan kritik yang bersifat membangun sangat diharapkan demi perbaikan di masa yang akan datang. Harapannya, laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembelajaran dalam bahasa pemrograman Java.

# BAB 1

# PENDAHULUAN

## 1.1 Latar Belakang

Abstraction memisahkan kontrak yang digunakan oleh pemanggil dari rincian implementasinya. Pada proyek toko online, Product menjadi dasar produk fisik dan digital, sedangkan PaymentMethod dan Shippable menyatakan kemampuan pembayaran dan pengiriman. CheckoutService dapat memproses produk serta metode pembayaran melalui kontrak tersebut tanpa bergantung pada setiap class konkret.

## 1.2 Tujuan Praktikum

Praktikum ini bertujuan menerapkan abstract class, interface, default method, static method, private method, interface inheritance, final, dan nested class dalam Java. Tugas lanjutannya menerapkan evolusi API biaya admin, penyelesaian konflik default method, sistem notifikasi, serta review dan refactor desain BadShop.

## 1.3 Manfaat Praktikum

Penerapan kontrak memungkinkan penambahan kemampuan tanpa mengubah seluruh pemanggil. Rumus bersama dapat dipertahankan melalui final method, data alamat dibuat immutable, dan implementasi kanal baru dapat didaftarkan melalui NotificationChannel. Hasilnya dapat diperiksa melalui SelfCheck dan TugasCheck.

# BAB 2

# PEMBAHASAN

## 2.1 Implementasi Latihan Abstraction

Product adalah abstract class dengan data SKU, nama, dan harga. PhysicalProduct mengimplementasikan Shippable serta menolak berat kurang dari atau sama dengan nol. DigitalProduct mengimplementasikan Discountable dengan diskon 10 persen sehingga harga e-book Rp50000 menjadi Rp45000. Perhitungan subtotal menggunakan getFinalPrice() sehingga diskon dipanggil secara polimorfik.

PaymentMethod menjadi kontrak getName() dan pay(long). EWalletPayment mengurangi saldo saat pembayaran berhasil. BankTransferPayment menyimulasikan pembayaran untuk nominal positif. CreditCardPayment mengimplementasikan RefundablePayment dan InstallmentPayment sekaligus. Cicilan memakai pembulatan ke atas dan dibatasi maksimal 12 bulan.

ShippingService memusatkan rumus ongkir pada calculateCost() yang final; subclass hanya menentukan ratePerKg(). Berat 700 gram dibulatkan menjadi satu kilogram. Address adalah final class dengan field private final tanpa setter. Order.Line dibuat static nested, Order.Receipt dibuat inner untuk membaca data pesanan, dan voucher ShopApp menggunakan anonymous class.

Keranjang contoh berisi kopi Rp85000, rendang Rp120000, dan e-book setelah diskon Rp45000. Subtotalnya Rp250000, berat kiriman 700 gram, total Reguler Rp260000, dan total Ekspres Rp268000 sebelum biaya admin.

## 2.2 Pemeriksaan Latihan dan Error Compiler

SelfCheck menghasilkan 23 dari 23 pemeriksaan lulus. Percobaan error dilakukan pada source sementara agar program akhir tetap dapat dikompilasi. Bukti berikut adalah transkrip eksekusi, bukan screenshot desktop.

**Percobaan L1**

Product abstrak belum memberikan implementasi category(), sehingga tidak dapat diinstansiasi langsung.

```
[folder sementara]/Probe.java:1: error: Product is abstract; cannot be
instantiated
class Probe { Object p = new shop.model.Product("X", "Uji", 1000); }
                         ^
1 error
```

**Percobaan L3**

Parameter refund harus RefundablePayment; BankTransferPayment hanya PaymentMethod sehingga ditolak pada waktu kompilasi.

```
[folder sementara]/Probe.java:1: error: incompatible types:
BankTransferPayment cannot be converted to RefundablePayment
class Probe { void test() { new shop.service.CheckoutService().refund(new
 shop.payment.BankTransferPayment("Bank Nagari"), 1000); } }
                                                                      ^
Note: Some messages have been simplified; recompile with -Xdiags:verbose
to get full output
1 error
```

**Percobaan L4**

calculateCost final tidak dapat di-override. Kurir mengubah ratePerKg, bukan rumus pembulatan bersama.

```
[folder sementara]/Probe.java:1: error: calculateCost(int) in Probe
cannot override calculateCost(int) in ShippingService
class Probe extends shop.shipping.ShippingService { Probe() {
super("Gratis"); } protected long ratePerKg() { return 0; } public long
calculateCost(int grams) { return 0; } }

                                                          ^
  overridden method is final
1 error
```

**Hasil SelfCheck**

```
=== SELF CHECK: Praktikum Abstraction ===
```

```
-- L1 - Abstract class & abstract method
  [LULUS] L1.1 Product adalah abstract class
  [LULUS] L1.2 Product punya abstract method category()
  [LULUS] L1.3 PhysicalProduct & DigitalProduct adalah subclass konkret
Product
  [LULUS] L1.4 Konstruktor menolak harga negatif
(IllegalArgumentException)
  [LULUS] L1.5 getInfo() PhysicalProduct sesuai format
  [LULUS] L1.6 ShippingService abstract dengan abstract method
ratePerKg()
  [LULUS] L1.7 calculateCost(): Reguler 700g=10000, 1001g=20000, 0g=0;
Ekspres 700g=18000

-- L2 - Interface & program to an interface
  [LULUS] L2.1 Shippable interface; PhysicalProduct implements,
DigitalProduct tidak
  [LULUS] L2.2 PaymentMethod interface dengan getName() dan pay(long)
  [LULUS] L2.3 EWalletPayment: bayar sukses mengurangi saldo, saldo
kurang ditolak
  [LULUS] L2.4 CheckoutService.checkout(...) menerima PaymentMethod &
ShippingService (bukan class konkret)
  [LULUS] L2.5 Total keranjang contoh: Reguler=260000, Ekspres=268000

-- L3 - Default, static, private method; interface inheritance
  [LULUS] L3.1 Discountable: applyDiscount default, percentOf static, ada
 private method
  [LULUS] L3.2 DigitalProduct implements Discountable; harga akhir 50000
-> 45000
  [LULUS] L3.3 RefundablePayment & InstallmentPayment adalah interface
turunan PaymentMethod
  [LULUS] L3.4 CreditCardPayment implements RefundablePayment DAN
InstallmentPayment
  [LULUS] L3.5 monthlyInstallment(100000, 3)=33334 dan 13 bulan ditolak
  [LULUS] L3.6 CheckoutService.refund(...) hanya menerima
RefundablePayment

-- L4 - final method & final class
  [LULUS] L4.1 ShippingService.calculateCost(int) adalah final method
  [LULUS] L4.2 Address final class, semua field private final, tanpa
setter

-- L5 - Static nested, inner, dan anonymous class
  [LULUS] L5.1 Order.Line static nested; Order.Receipt inner (non-static)
  [LULUS] L5.2 Receipt.toText() memuat id pesanan, total barang, dan kota
 tujuan
  [LULUS] L5.3 ShopApp memakai anonymous class yang mengimplementasikan
PaymentMethod

=== HASIL: 23 / 23 pemeriksaan lulus ===
```

# BAB 3

# TUGAS

## 3.1 Evolusi API dan Konflik Diamond

### Parameter Pribadi Berdasarkan NIM

```
Parameter tugas untuk NIM 2511532001
  CARD_ADMIN_PERCENT  = 2
  TRANSFER_ADMIN_FEE  = 3000
  SMS_MAX_LENGTH      = 101
```

NIM 2511532001 memiliki digit terakhir 1 dan dua digit terakhir 01. CARD_ADMIN_PERCENT = 2, TRANSFER_ADMIN_FEE = Rp3000, dan SMS_MAX_LENGTH = 101. Identitas dan ketiga konstanta disimpan pada StudentInfo.

T1.a menghasilkan 4 error untuk keadaan L3: EWalletPayment, BankTransferPayment, CreditCardPayment, dan anonymous class voucher di ShopApp. Method abstract yang baru menjadi kewajiban semua implementasi konkret. Anonymous class juga class implementasi, sehingga tetap wajib menyediakan adminFee(long). Percobaan dilakukan pada salinan source keadaan sebelum override biaya admin; proyek akhir tetap dapat dikompilasi.

Default adminFee mengembalikan 0 sehingga implementasi lama tetap sah. Kartu kredit meng-override dengan persentase dari StudentInfo, transfer memakai biaya tetap, dan e-wallet mewarisi 0. Checkout membayar grandTotal: total produk dan ongkir ditambah biaya admin untuk total tersebut.

EWalletPayment hanya mewarisi satu policy dari RefundablePayment, sehingga tidak mengalami konflik. CreditCardPayment mewarisi dua default policy dari interface yang tidak saling lebih spesifik. Java tidak dapat menebak kebijakan bisnis yang dimaksud; class harus memilih atau menggabungkannya secara eksplisit. Override menggunakan RefundablePayment.super.policy() + "; " + InstallmentPayment.super.policy().

| **Metode (Ekspres)** | **Total sebelum admin** | **Admin** | **Grand total** |
|---|---|---|---|
| E-wallet | Rp268000 | Rp0 | Rp268000 |
| Transfer | Rp268000 | Rp3000 | Rp271000 |
| Kartu kredit | Rp268000 | Rp5360 | Rp273360 |

**Error Compiler T1a**

Ada 4 error pada tiga metode pembayaran dan anonymous voucher. Semuanya belum menyediakan method abstract adminFee yang baru.

```
[folder sementara]/T1a\shop\app\ShopApp.java:65: error: <anonymous
shop.app.ShopApp$1> is not abstract and does not override abstract method
 adminFee(long) in PaymentMethod
        PaymentMethod voucher = new PaymentMethod() { // anonymous class
                                                      ^
[folder sementara]/T1a\shop\payment\BankTransferPayment.java:5: error:
BankTransferPayment is not abstract and does not override abstract method
 adminFee(long) in PaymentMethod
public class BankTransferPayment implements PaymentMethod {
       ^
[folder sementara]/T1a\shop\payment\CreditCardPayment.java:5: error:
CreditCardPayment is not abstract and does not override abstract method
adminFee(long) in PaymentMethod
public class CreditCardPayment implements RefundablePayment,
InstallmentPayment {
       ^
[folder sementara]/T1a\shop\payment\EWalletPayment.java:3: error:
EWalletPayment is not abstract and does not override abstract method
adminFee(long) in PaymentMethod
public class EWalletPayment implements RefundablePayment {
       ^
4 errors
```

**Error Compiler T1e**

CreditCardPayment mewarisi dua default policy yang tidak saling lebih spesifik. Override eksplisit diperlukan untuk menggabungkannya.

```
[folder sementara]/T1e\shop\payment\CreditCardPayment.java:5: error:
types RefundablePayment and InstallmentPayment are incompatible;
public class CreditCardPayment implements RefundablePayment,
InstallmentPayment {
       ^
  class CreditCardPayment inherits unrelated defaults for policy() from
types RefundablePayment and InstallmentPayment
1 error
```

### 3.2 Sistem Notifikasi Pesanan

Interface NotificationChannel menyatakan kemampuan mengirim. AbstractChannel menyimpan penghitung dan pesan terakhir serta menyediakan urutan pengiriman bersama. Keduanya dibutuhkan agar kanal biasa bisa memakai ulang algoritma, sementara kanal log sekali pakai tetap dapat mengimplementasikan kontrak langsung.

send dibuat final agar subclass tidak melewati validasi, penyiapan pesan, atau pencatatan. Subclass hanya mengisi resolveRecipient dan deliver, serta boleh mengganti prepare. Jika send dapat di-override, kanal dapat melaporkan berhasil tanpa memperbarui penghitung atau mengirim pesan yang belum divalidasi.

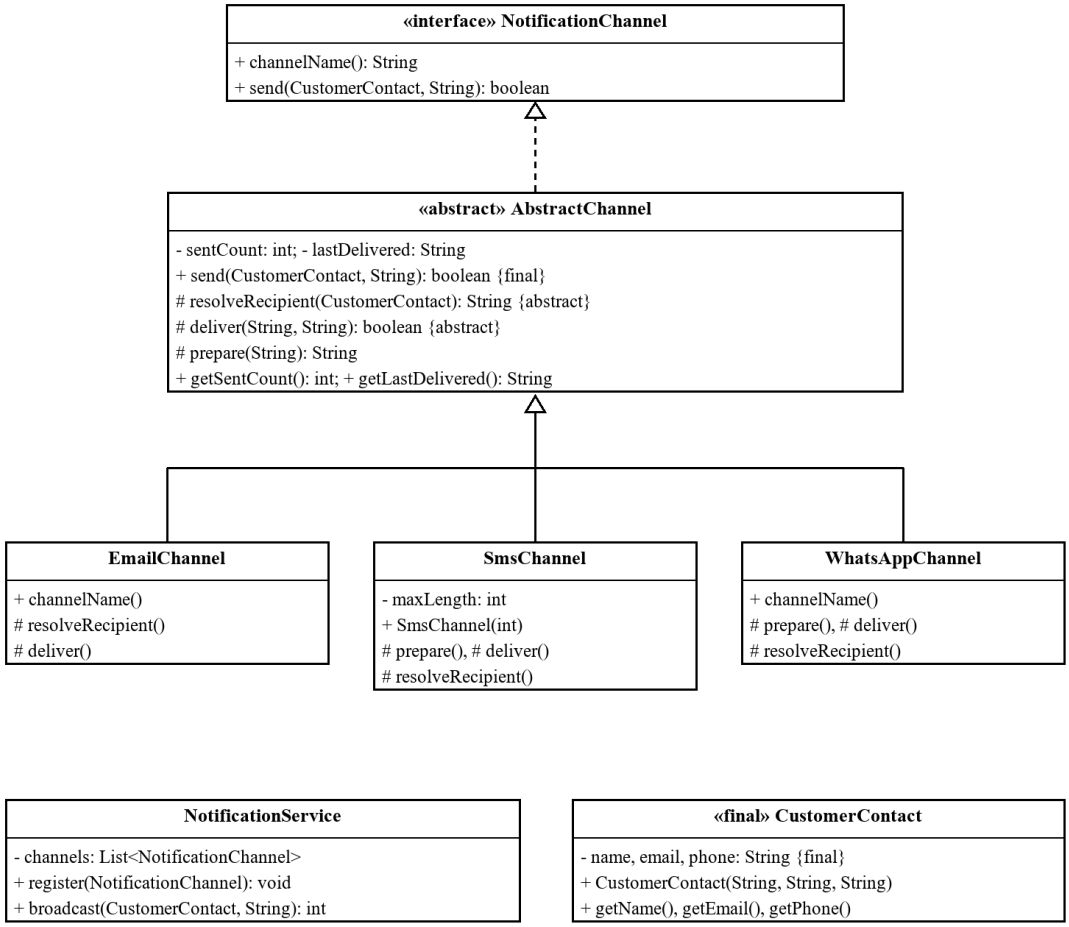
NotificationService menyimpan List<NotificationChannel> agar menerima seluruh implementasi kontrak, termasuk anonymous class log yang bukan AbstractChannel. Kanal baru cukup didaftarkan, tanpa mengubah broadcast. Pengiriman di tugas ini berupa simulasi cetak konsol, bukan akses layanan email/SMS/WhatsApp sungguhan.

CustomerContact merupakan final class; name wajib, sedangkan null pada email dan phone diubah menjadi string kosong. EmailChannel menerima alamat yang mengandung @. SmsChannel menerima nomor berawalan 08 dan memotong pesan

menjadi tepat 101 karakter termasuk tiga titik. WhatsAppChannel menerima nomor 08 atau +62 dengan panjang minimal 10 serta menambahkan awalan [Toko UMKM].

TugasApp mendaftarkan tiga kanal tersebut dan satu kanal log berupa anonymous class NotificationChannel. Pesan lebih dari 200 karakter dikirim kepada Siti Rahma yang memiliki email dan nomor 08 serta Budi yang hanya memiliki nomor +62. Hasilnya Siti menerima melalui empat kanal, sedangkan Budi menerima melalui dua kanal.

## Diagram UML Sistem Notifikasi




NotificationService o── 0..* NotificationChannel
NotificationService dan NotificationChannel ..> CustomerContact

Garis segitiga putus-putus menyatakan implements, sedangkan garis segitiga utuh menyatakan extends. NotificationService menyimpan nol atau lebih NotificationChannel; interface dan service memakai CustomerContact sebagai parameter. Sumber diagram yang dapat diedit tersedia pada notification.puml.

## Hasil TugasApp

```
Tugas oleh Az Zahrand Solichul Tajussalathin (2511532001)

== T1: Total dengan biaya admin (Ekspres) ==
E-Wallet DompetKu : Rp268000 (admin Rp0)
Transfer Bank Nagari : Rp271000 (admin Rp3000)
Kartu Kredit (Siti Rahma) : Rp273360 (admin Rp5360)

Kebijakan kartu kredit : Refund maksimal 7 hari; Cicilan maksimal 12
bulan
Kebijakan e-wallet     : Refund maksimal 7 hari

== T2: Notifikasi status pesanan ==
[EMAIL to siti@example.com] Pesanan ORD-0001 sudah dikirim dengan kurir
Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasih
telah berbelanja produk UMKM Sumatera Barat di toko kami! Silakan periksa
 alamat tujuan dan hubungi layanan pelanggan jika membutuhkan bantuan.
[SMS to 081234567890] Pesanan ORD-0001 sudah dikirim dengan kurir
Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24...
[WA to 081234567890] [Toko UMKM] Pesanan ORD-0001 sudah dikirim dengan
kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima
kasih telah berbelanja produk UMKM Sumatera Barat di toko kami! Silakan
periksa alamat tujuan dan hubungi layanan pelanggan jika membutuhkan
bantuan.
[LOG] Siti Rahma: Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres.
Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasih telah
berbelanja produk UMKM Sumatera Barat di toko kami! Silakan periksa
alamat tujuan dan hubungi layanan pelanggan jika membutuhkan bantuan.
Terkirim ke Siti Rahma lewat 4 kanal
[WA to +6281298765432] [Toko UMKM] Pesanan ORD-0001 sudah dikirim dengan
kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima
kasih telah berbelanja produk UMKM Sumatera Barat di toko kami! Silakan
periksa alamat tujuan dan hubungi layanan pelanggan jika membutuhkan
bantuan.
[LOG] Budi: Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor
resi akan muncul di aplikasi dalam 1x24 jam. Terima kasih telah
berbelanja produk UMKM Sumatera Barat di toko kami! Silakan periksa
alamat tujuan dan hubungi layanan pelanggan jika membutuhkan bantuan.
Terkirim ke Budi lewat 2 kanal
```

## Hasil TugasCheck

```
=== TUGAS CHECK: Praktikum Abstraction ===
NIM: 2511532001 | Az Zahrand Solichul Tajussalathin

-- Parameter NIM
  [LULUS] P.1 Konstanta di StudentInfo sesuai NIM (2%, Rp3000, 101
karakter)
```

```
-- T1 - Evolusi API & diamond
  [LULUS] T1.1 PaymentMethod.adminFee(long) adalah default method;
e-wallet = 0
  [LULUS] T1.2 Biaya admin kartu kredit & transfer sesuai parameter NIM
  [LULUS] T1.3 grandTotal(...) = total + biaya admin (kartu kredit,
keranjang contoh, Ekspres)
  [LULUS] T1.4 Diamond policy(): kartu kredit menggabungkan kedua
kebijakan

-- T2 - Sistem notifikasi
  [LULUS] T2.1 NotificationChannel interface; AbstractChannel abstract
dengan send(...) final
  [LULUS] T2.2 EmailChannel, SmsChannel, WhatsAppChannel adalah subclass
AbstractChannel
[SMS to 081234567890] xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx...
  [LULUS] T2.3 SmsChannel memotong pesan panjang tepat 101 karakter
berakhiran "..."
[EMAIL to uji@example.com] Halo
  [LULUS] T2.4 EmailChannel menolak kontak tanpa email yang valid
[WA to +6281298765432] [Toko UMKM] Pesanan dikirim
  [LULUS] T2.5 NotificationService.broadcast(...) mengembalikan jumlah
kanal yang berhasil

=== HASIL: 10 / 10 pemeriksaan lulus ===
```

### 3.3 Code Review dan Refactor BadShop

| Lokasi | Masalah | Dampak nyata | Perbaikan |
|---|---|---|---|
| StoreOperations / GiftVoucher | Kontrak mencampur pembayaran, refund, pengiriman, struk. | ship/refund voucher melempar UnsupportedOperationException. | Voucher hanya implements PaymentMethod. |
| Courier.cost | Switch String mengenali jenis kurir; typo EXPRES masuk default. | Ongkir 2 kg menjadi 0; kanal baru perlu ubah switch. | ShippingService dan ExpressShipping; rumus final. |
| Courier / JneCourier | Abstract class tanpa variasi abstrak; subclass kosong. | Pewarisan tidak memberi perilaku kurir. | ratePerKg abstrak, calculateCost final. |

| Lokasi | Masalah | Dampak nyata | Perbaikan |
|---|---|---|---|
| Address / SimpleOrder | Alamat mutable dibagi dua pesanan; final hanya referensi. | Mengubah ORD-2 ikut mengubah ORD-1. | Address final, field final, tanpa setter; alamat baru untuk ORD-2. |
| Cart.Item | Inner class tanpa kebutuhan mengakses Cart. | Menyimpan referensi luar; konstruksi bergantung objek Cart. | Jadikan static nested; new Cart.Item(...). |
| Discount.apply | Default method memakai instanceof class konkret. | Promo baru perlu edit interface; kontrak bergantung implementasi. | Discountable.discountPercent dan default applyDiscount. |
| GiftVoucher.pay | Uang double; amount negatif tidak ditolak. | pay(-1000) justru menambah saldo; representasi uang tidak konsisten. | Saldo long; tolak amount <= 0 dan saldo kurang. |

| Lokasi | Masalah | Dampak nyata | Perbaikan |
|---|---|---|---|
| Cart.Item / Cart.add | Tidak memvalidasi jumlah, harga, nama, null. | Subtotal negatif atau NullPointerException saat total. | Validasi constructor dan add sebelum menyimpan. |

Refactor tersimpan terpisah pada src/review/fixed/BadShopRefactored.java. File BadShop.java asli tidak diubah. Versi refactor memakai ulang ShippingService, ExpressShipping, Discountable, dan PaymentMethod dari proyek. Semua gejala utama pada demo hilang.

```
Total cart: 170000
Ongkir Ekspres 2 kg: 36000
ORD-1 -> Jl. Sudirman 10, Padang
ORD-2 -> Jl. Sudirman 10, Bukittinggi
Harga flash sale: 70000
true | sisa Rp70000
```

### 3.4 Refleksi Pilihan Desain

| Kondisi | Pilihan | Alasan dan contoh proyek |
|---|---|---|
| Berbagi field dan constructor | Abstract class | Product memusatkan sku, name, price dan validasi constructor untuk produk fisik/digital. |

| **Kondisi** | **Pilihan** | **Alasan dan contoh proyek** |
|---|---|---|
| Beberapa kemampuan sekaligus | Interface | CreditCardPayment implements RefundablePayment dan InstallmentPayment. |
| Rumus bersama tidak boleh diganti | Abstract class + final method | ShippingService.calculateCost final mengunci pembulatan; ratePerKg tetap abstrak. |
| Kontrak berkembang | Interface + default method | PaymentMethod.adminFee default 0 menjaga implementasi lama tetap dapat dikompilasi. |

| **Kondisi** | **Pilihan** | **Alasan dan contoh proyek** |
|---|---|---|
| Pembantu tanpa data objek luar | Static nested | Order.Line memakai product dan quantity sendiri; dapat dibuat tanpa Order. |
| Perlu membaca field objek luar | Inner | Order.Receipt membaca Order.this.orderId dan shippingAddress. |
| Implementasi sekali pakai | Anonymous | TugasApp mendaftarkan kanal log konsol melalui new NotificationChannel() { ... }. |

# BAB 4

# KESIMPULAN

## 4.1 Ringkasan Hasil Praktikum

Abstraction pada proyek toko online diwujudkan melalui Product dan ShippingService sebagai abstract class serta PaymentMethod, Shippable, Discountable, dan NotificationChannel sebagai interface. Default method memungkinkan penambahan biaya admin tanpa mewajibkan seluruh implementasi lama diubah. Konflik dua default policy diselesaikan secara eksplisit pada CreditCardPayment.

Algoritma pengiriman notifikasi dikunci melalui send() yang final dan dilengkapi hook untuk tiap kanal. Address menjaga data alamat melalui field private final tanpa setter. Refactor BadShop menghilangkan ongkir Ekspres nol, perubahan alamat pesanan lain, dan exception pengiriman voucher. Verifikasi yang sudah dijalankan menghasilkan SelfCheck 23/23 dan TugasCheck 10/10 pemeriksaan lulus.

## 4.2 Link Github

https://github.com/Zarnnonly/PrakPBO_A_2511532001

## DAFTAR PUSTAKA

[1] Program Studi S1 Informatika Universitas Andalas. Modul Praktikum OOP 6 – Abstraction. Bagian C–E, halaman 6–22.

[2] Az Zahrand Solichul Tajussalathin. Kode sumber proyek abstraction-2511532001 dan hasil eksekusi SelfCheck, TugasCheck, serta TugasApp. NIM 2511532001.